

Java Swing 程式碼錯誤分析

<https://gemini.google.com/app/14f3c7eea04b7ff1>

User prompt: 這段程式碼哪裡有問題，說明一下

Response: 這段 Java Swing 程式碼主要有以下幾個顯著問題：

1. 致命錯誤：字串比較使用了 ==

- **問題：**`if (t1.getText() == "admin" && t2.getText() == "...")`
- **原因：**在 Java 中，`==` 比較的是物件的記憶體位址（Reference），而不是字串內容。即使輸入框內填入 "admin"，`t1.getText() == "admin"` 依然會傳回 `false`。
- **修正：**應使用 `.equals()` 方法：

```
if ("admin".equals(t1.getText()) && "密碼內容".equals(t2.getText()))
```

2. 介面呈現問題：使用 null 佈局但未設定元件位置與大小

- **問題：**程式碼寫了 `setLayout(null);`，但建立 `JTextField`、`JLabel` 與 `JButton` 後直接 `add()`，完全沒有呼叫 `.setBounds(x, y, width, height)`。
- **結果：**所有元件預設的大小與座標都是 0，畫面上會完全看不到任何輸入框或按鈕（畫面一片空白）。
- **修正：**需為每個元件設定位置（例如 `t1.setBounds(50, 50, 100, 30);`），或是改用合理的版面配置管理器（如 `FlowLayout` 或 `GridLayout`）。

3. 畫面顯示時機問題：setVisible(true) 過早呼叫

- **問題：**在建構子初期就執行了 `setVisible(true);`，而元件是在這之後才 `add()` 進去。
- **結果：**視窗彈出時元件還沒被加載，通常需要調整視窗大小或手動刷洗才能顯示出來。
- **修正：**應將 `setVisible(true);` 移至所有元件都已 `add()` 到視窗的最後一行。

4. 資安問題：密碼使用明文 JTextField

- **問題：**密碼欄位 `t2` 使用了一般的 `JTextField`。
- **結果：**使用者輸入密碼時會直接顯示明文字元，無法隱藏（變成星號或圓點）。
- **修正：**密碼輸入框應改用 `JPasswordField`。

5. 拼字錯誤 (Typo)

- **問題：**匯入套件時寫成 `inport javax.swing.*;`。

- **結果：**應為 `import`，這會直接導致編譯失敗。
-

User prompt: 修正成正確的完整

Response: 以下是修正後完整且可正常執行的 Java 程式碼：

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class CorrectLogin extends JFrame {

    public CorrectLogin() {
        // 1. 設定視窗基本屬性
        setTitle("登入");
        setSize(300, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // 關閉視窗時結束程式
        setLocationRelativeTo(null); // 讓視窗顯示在螢幕中央

        // 2. 使用版面配置管理器 (Layout Manager)，避免元件看不到
        setLayout(new GridLayout(3, 2, 10, 10));

        // 3. 建立 UI 元件 (密碼欄位改用 JPasswordField)
        JLabel l1 = new JLabel("帳號:", SwingConstants.CENTER);
        JTextField t1 = new JTextField();

        JLabel l2 = new JLabel("密碼:", SwingConstants.CENTER);
        JPasswordField t2 = new JPasswordField(); // 隱藏密碼輸入內容

        JButton btn = new JButton("登入");
        JLabel resultLabel = new JLabel("", SwingConstants.CENTER); // 用於顯示結果

        // 4. 將元件加入視窗
        add(l1);
        add(t1);
        add(l2);
        add(t2);
        add(btn);
        add(resultLabel);

        // 5. 設定按鈕事件監聽器 (修復字串比較的問題)
        btn.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                String username = t1.getText();
                String password = new String(t2.getPassword()); // 取得密碼字串

                // 正確使用 .equals() 進行字串比較，而非 ==
            }
        });
    }
}
```

```
        if ("admin".equals(username) && "1234".equals(password)) {
            System.out.println("登入成功");
            resultLabel.setText("登入成功!");
            resultLabel.setForeground(Color.GREEN);
        } else {
            System.out.println("帳號或密碼錯誤");
            resultLabel.setText("帳號或密碼錯誤");
            resultLabel.setForeground(Color.RED);
        }
    }
});

// 6. 最後才顯示視窗，確保所有元件已載入
setVisible(true);
}

public static void main(String[] args) {
    // Swing 建議在 Event Dispatch Thread (EDT) 中建立 UI
    SwingUtilities.invokeLater(() -> new CorrectLogin());
}
}
```

主要修正與改進重點：

1. **修正 import**：將 `inport` 改為 `import`。
2. **修正字串比較**：改用 `"admin".equals(username)` 比較字串內容，而不是用 `==` 比較位址。
3. **設定版面配置 (Layout)**：採用 `GridLayout(3, 2, 10, 10)` 自動排版，解決原先沒有指定元件大小位置導致畫面一片空白的問題。
4. **密碼安全性**：將 `t2` 的類型由 `JTextField` 改為 `JPasswordField`。
5. **setVisible(true) 位置**：移至建構子的最後一行，確保元件完整載入後才呈現視窗。
6. **視窗關閉行為**：補上 `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`，讓按下關閉鈕時能正常結束程式。

User prompt: 帳密是什麼

Response: 修正後的程式碼中設定的預設預設預設帳號與密碼如下：

- 帳號：admin
- 密碼：1234

程式碼中的驗證判斷式如下：

```
if ("admin".equals(username) && "1234".equals(password))
```

User prompt: 解釋一下程式碼

Response: 這份修正後的 Java Swing 程式碼主要分為 **視窗基本設定**、**版面配置與元件建立**、**按鈕點擊事件處理** 以及 **主程式進入點** 四個核心部分：

1. 視窗基本設定（建構子開頭）

```
setTitle("登入");  
setSize(300, 200);  
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
setLocationRelativeTo(null);
```

- **setTitle & setSize**：設定視窗標題列顯示的文字為「登入」，並將視窗寬高設為 300×200 像素。
- **setDefaultCloseOperation**：設定使用者按下視窗右上角的 X 關閉按鈕時，會直接結束整個 Java 應用程式（否則視窗關閉後背景程式可能還在執行）。
- **setLocationRelativeTo(null)**：傳入 null 可以讓視窗自動顯示在螢幕的正中央。

2. 版面配置與 UI 元件建立

```
setLayout(new GridLayout(3, 2, 10, 10));  
  
JLabel l1 = new JLabel("帳號:", SwingConstants.CENTER);  
JTextField t1 = new JTextField();  
  
JLabel l2 = new JLabel("密碼:", SwingConstants.CENTER);  
JPasswordField t2 = new JPasswordField();  
  
JButton btn = new JButton("登入");  
JLabel resultLabel = new JLabel("", SwingConstants.CENTER);
```

- **setLayout(new GridLayout(3, 2, 10, 10))**：採用網格版面配置，設定為 **3 列 2 欄**，元件之間的水平與垂直間距皆為 10 像素。元件會依序從左到右、從上到下自動填滿網格。
- **JLabel**：文字標籤，用來顯示提示訊息（如「帳號:」與「密碼:」）。`SwingConstants.CENTER` 讓文字居中顯示。
- **JTextField**：單行文字輸入框，讓使用者輸入帳號。
- **JPasswordField**：專用密碼輸入框，使用者輸入的字元會自動轉為黑點或星號遮罩，保障隱私。
- **JButton**：可點擊的「登入」按鈕。
- **resultLabel**：預設為空的標籤，用來在畫面下方動態顯示登入成功或失敗的訊息。

3. 將元件加入視窗與顯示

```
add(l1); add(t1);
add(l2); add(t2);
add(btn); add(resultLabel);

setVisible(true);
```

- **add(...)**：依序將建立好的元件放入 GridLayout 的 6 個格子中。
- **setVisible(true)**：放在建構子最後一行，確保所有元件都已經載入與排版完畢後，才一次性繪製並顯示視窗。

4. 按鈕點擊事件監聽（核心邏輯）

```
btn.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        String username = t1.getText();
        String password = new String(t2.getPassword());

        if ("admin".equals(username) && "1234".equals(password)) {
            System.out.println("登入成功");
            resultLabel.setText("登入成功！");
            resultLabel.setForeground(Color.GREEN);
        } else {
            System.out.println("帳號或密碼錯誤");
            resultLabel.setText("帳號或密碼錯誤");
            resultLabel.setForeground(Color.RED);
        }
    }
});
```

- **addActionListener**：給按鈕註冊一個事件監聽器，當使用者點擊「登入」按鈕時會自動觸發 actionPerformed 方法。
- **取得輸入內容**：
 - **t1.getText()**：讀取帳號輸入框的文字。
 - **t2.getPassword()**：密碼框會傳回字元陣列 char[]，使用 new String(...) 將其轉為字串。
- **equals() 判斷**：使用 "admin".equals(username) 比較字串內容。若帳號為 admin 且密碼為 1234：
 - **成功**：resultLabel 顯示綠色的「登入成功！」。
 - **失敗**：resultLabel 顯示紅色的「帳號或密碼錯誤」。

5. 主程式進入點

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new CorrectLogin());
}
```

```
}
```

- **SwingUtilities.invokeLater**：Swing 框架**非線程安全（Not Thread-Safe）**，將建立視窗的程式碼放到 Event Dispatch Thread (EDT) 事件處理執行緒中執行，是撰寫 Java GUI 的最佳標準實作。
-